

6. Optimización

Optimización (Búsquedas locales)

$$f : X \rightarrow \mathbb{R}$$

Lo que quiero encontrar es una x^* tal que:

$$\begin{aligned} f(x^*) &= \max_{x \in X} f(x) \\ f(x^*) &= \min_{x \in X} -f(x) \end{aligned}$$

El tipo de problema más famoso que requiere optimización es el *Problema del Agente Viajero*. Teniendo una matriz A que representa un grafo de viaje, queremos encontrar un camino ciclado x (o permutación) de $[1, \dots, n]$ donde:

$$f(x) = \sum_{i=1}^{n-1} a_{x[i], x[i+1]} + a_{x[n], x[1]}$$

Esto no se puede hacer con Dykstra. Con camino ciclado me refiero a algo así:

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ 10 & 1 & 4 & 5 & 6 & 9 & 3 & 8 & 2 & 7 \end{bmatrix}$$

Cuando tenemos estos problemas "optimizar" no significa encontrar el mejor siempre o el mínimo total o el máximo total. Aquí entra otra nomenclatura.

Si $f : X \rightarrow \mathbb{R}$, un máximo global es:

$$X^\forall = \arg \max_{x \in X} f(x)$$

Si existe una topología en X (no se puede medir una distancia, pero se puede medir una "vecindad"), un máximo local es:

$$x^* = \arg \max_{x \in \text{Vecinos}(x)} f(x)$$

Por ejemplo, Tenemos una tabla de ajedrez que 4x4 en donde se colocan 4 reinas. Entre más posibilidades existan de que una reina se coma a otra, el costo se le resta -1.

```
class PbOptim()
    def costo(self, x):
        if not self.es_estado(x):
            raise ValueError('---')
        return f(x)

    def es_estado(self, x):
```

```

        return True/False

def estado_aleatorio(self):
    return estado

def vecinos(self,x):
    return iterable con los estados vecinos de x

def vecino_aleatorio(self,x):
    return vecino de x

# Modelo Dummy
def BusquedaAleatoria(pb, num_iteraciones):
    x = pb.estado_aleatorio()
    c = pb.costo(x)

    for _ in range(num_iter):
        otro = pb.estado_aleatorio()
        c_otro = pb.costo(otro)

        if c_otro < c:
            x, c = otro, c_otro

# Para maximizar
def descenso_colinas(pb, num_iteraciones):
    x = pb.estado_aleatorio()
    c = pb.costo(x)

    for _ in range(num_iter):
        termina = True

        for vecino in pb.vecinos(x):
            c_vecino = pb.costo(vecino)

            if c_vecino < c:
                x, c = vecino, c_vecino
                terminal False

        if terminal:
            break

    return x, c

# Simulated Annealing (Metodo metaheurístico)
def temple_simulado(pb, Tmax, calendarizador, Tmin):
    x = pb.estado_aleatorio()

```

```

c = pb.costo(x)

T= Tmax
i = 0

while (T > Tmin):
    vecino = pb.vecino_aleatorio(x)
    c_vecino = pb.costo(vecino)

    incremento = c - c_vecino

    if incremento >= 0 or random.random() < math.exp(incremento/T):
        x, c = vecino, c_vecino

    T = calendarizador(T, i)
    i += 1

return x, c

def calendarizador(Tmax, i):
    return Tmax/(math.log(i+1)+1)

```

Teoría del sexo

Imagina que tienes una lista de puntos u objetos, a este le llamamos población:

$$\text{Población} = \{x_1, x_2, \dots, x_n\} \quad x_i \in X$$

Cada punto va a tener una lista de tamaño fijo de características numéricas o numéricas:

$$\text{Características}[V_{x_n}, V_{x_n}, \dots, V_{i_n}]$$

Ahora necesitamos una función de *adaptación* la cual indica las probabilidades de que el objeto sobreviva:

$$\text{Adaptación} : X \rightarrow \mathbb{R} \quad \text{Adaptación}(x_i)$$

Esta función es inversamente proporcional al costo. Se puede calcular así de las siguientes formas y puedes elegir cualquiera según lo que prefieras:

$$\text{Adaptación}(x_i) = \frac{1}{\text{costo}(x_i)}$$

$$\text{Adaptación}(x_i) = K - \text{costo}(x_i)$$

$$\text{Adaptación}(x_i) = \text{Max}(0, k - \text{costo}(x_i))$$

Ahora necesitamos un método selección para ver cuál sobrevive a hacia la "adultez", principalmente hay 2.

- *Ruleta*: Los objetos con la adaptación mayor tiene mejores posibilidades de ser escogidos del set al azar. Cuando seleccionamos un objeto de manera aleatoria, se saca del set y se elige otro, formando una pareja. Ambos se devuelven al set al terminar y se forma otra pareja repitiendo el proceso.
- *Torneo*: Se organizan competencias entre los objetos del set y los que son más adaptados son los que se reproducen. Separas a los objetos entre dos grupos y eliges de manera aleatoria a uno de cada grupo para competir. El objeto de menor adaptación tiene una pequeña probabilidad de ganar, con el fin de asegurar variedad. Estas competencias se se dan hasta que existan dos finalistas, los cuales se reproducen. Esto se repite.

Con nuestros ganadores, hacemos una *cruza*. Aquí tomamos las características (visto arriba) de los objetos los cuales usamos para producir un objeto nuevo.

$$\begin{array}{rcl}
 \text{Objeto 1} & [V_{x_n}, V_{x_n}, \dots, V_{i_n}] \\
 \text{Objeto 2} & [V_{y_n}, V_{y_n}, \dots, V_{j_n}] \\
 \text{---} & \text{---} \\
 \text{Hijo} & [V_{h_n}, V_{h_n}, \dots, V_{h_n}]
 \end{array}$$

Puede lanzar una moneda por cada característica para ver que objeto pasa su ADN, puedes dividir la matriz de características para una mitad sea de un objeto y la otra mitad sea del otro objeto, etc.

Ahora es necesario aplicar *mutaciones* en las características de los objetos hijo para que exista algún tipo de aprendizaje. El número de mutaciones debe ser limitado y pocos. Consisten en ir por cada característica y aplicarle una probabilidad pequeña de cambiar aleatoriamente.

Finalmente, debe existir el *elitismo*. Esto se da cuando un objeto/punto tiene una adaptabilidad bastante grande, el cual le permite poder reproducirse en varias generaciones debido a su adaptabilidad. Los puntos de *elite* siempre compiten con otros puntos más *Jovenes* para derechos de reproducción.

Entonces.

1. Cada individuo se representan como cadenas.
2. Se hace una función de adaptación.
3. Se hace un operador de selección.
4. Se hace un operador de cruce.
5. Se hace un operador de mutación
6. Elitismo.

Repaso

Tenemos un:

$$f : X \rightarrow \mathbb{R}$$

y queremos un mínimo global:

$$x^* = \arg \min_x f(x)$$

x^* es un mínimo global si y solo si:

$$f(x^*) \leq f(x) \quad \forall x \in X$$

$\bar{x}$ es mínimo local si y solo si:

$$f(\bar{x}) \leq f(x) \quad \forall x \in \text{vecinos}(\bar{x})$$

La distribución de Boltzmann es utilizada también

$$P(e = \text{cambie}) = \begin{cases} 1 \\ e^{-\nabla e / \text{Temp}} \end{cases} \quad \text{si } \overline{W} < 0$$

Esto fue utilizado en el siguiente algoritmo:

- x es un estado.
- $c = f(x)$
- x' un vecino de x .
- $c' = f(x')$
- $T = \text{Temperatura}$
- $\nabla e = c - c'$
- Si $\nabla e < 0$:
 - $x \leftarrow x'$
- Si no, con probabilidad $e^{-\nabla e / T}$
 - $x \leftarrow x'$
- Regresa

```
class BusquedaLocal():  
    def costo(self, x)  
        return f(x)  
  
    def vecinos(self, x)  
        return 'Genereador de vecinos de x'
```

```
def valor_aleatorio(self)  
    return x
```